

ST115-HW2

James Holdaway

Packages

```
library(aRtsy)
```

Question #1

Install the aRtsy package. Show the code used to install the aRtsy package, but turn off the code chunk so it doesn't evaluate.

Next, explain the difference between `install.packages` and `library`

```
install.packages("aRtsy")
```

The code `install.packages("package-name")` installs the contents of a package to your computer. `library(package-name)` loads that package along with its functions, and data to your R session. In order to actually use a R package you must ensure that its been installed to your machine, then load it into the session using `library`.

Question #2

Create a level-1 section header at the top of your document and label it **Packages**. Next, under a Question-2 level two section header, make a code chunk and library the **aRtsy** package. We are going to use a function in this package to plot the Collatz conjecture. On your own, research and explain what the Collatz conjecture is. Then, write the Collatz conjecture equation using in-line LaTeX code.

```
library(aRtsy)
```

The Collatz conjecture is a question that asks if you can bring any number down to one following one of two processes.

1. If even: $(n/2)$
2. If odd: $(3n + 1)$

This process repeats with n being redefined as the output until $n = 1$.

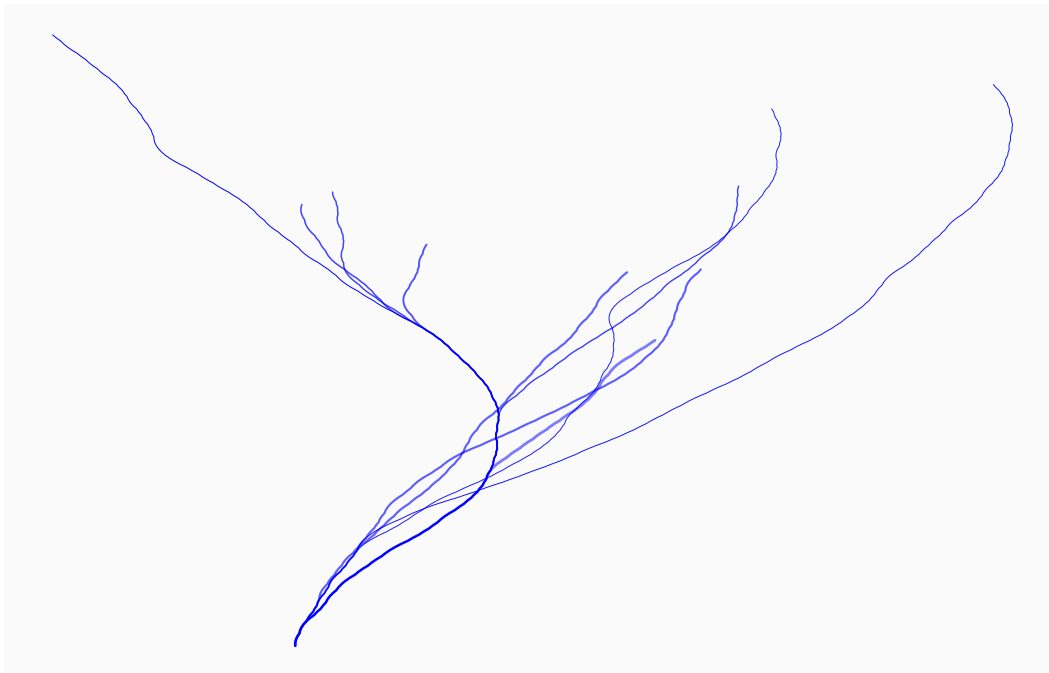
The conjecture can be represented like this:
$$f(n) = \begin{cases} \frac{n}{2} & \text{if } n \text{ is even} \\ 3n + 1 & \text{if } n \text{ is odd} \end{cases}$$

Question #3

Part A

Please see the following function `canvas_collatz()` from the `aRtsy` package in R. In 2-3 sentences, describe what this function is doing.

```
canvas_collatz(  
  colors = c("blue"),  
  n = 10  
)
```



When run, this function draws the Collatz conjecture on a canvas. The color argument specifies which color the artwork should be made in. The argument `n` specifies how many random starting integers should be used for the lines.

Part B

When you run the above code multiple times, you get different output. With this information...

- Explain why you get different out every time you run the code
- Explain why this might be troublesome when trying to put this code into your `.qmd` file

– Pull the help file up for `set.seed()`, and explain how this can be used to make sure we get the same result every time. Then, use the `set.seed()` function correctly with your favorite number, and include the code above in a code chunk

Each run produces a different output because `n = 10` means that 10 random integers are chosen as the starting value for the Collatz conjecture. Their nature of randomness means that they will be different each time, and consequently so will the graph of the conjecture.

This could be troublesome if you are analyzing a specific case of the Collatz conjecture. Random integers will not produce the same results.

The `set.seed()` function allows you to specify which seeds/integers you use as your starting point. This allows us to achieve the same result every time because we are specifying the starting point rather than having it chosen for us.

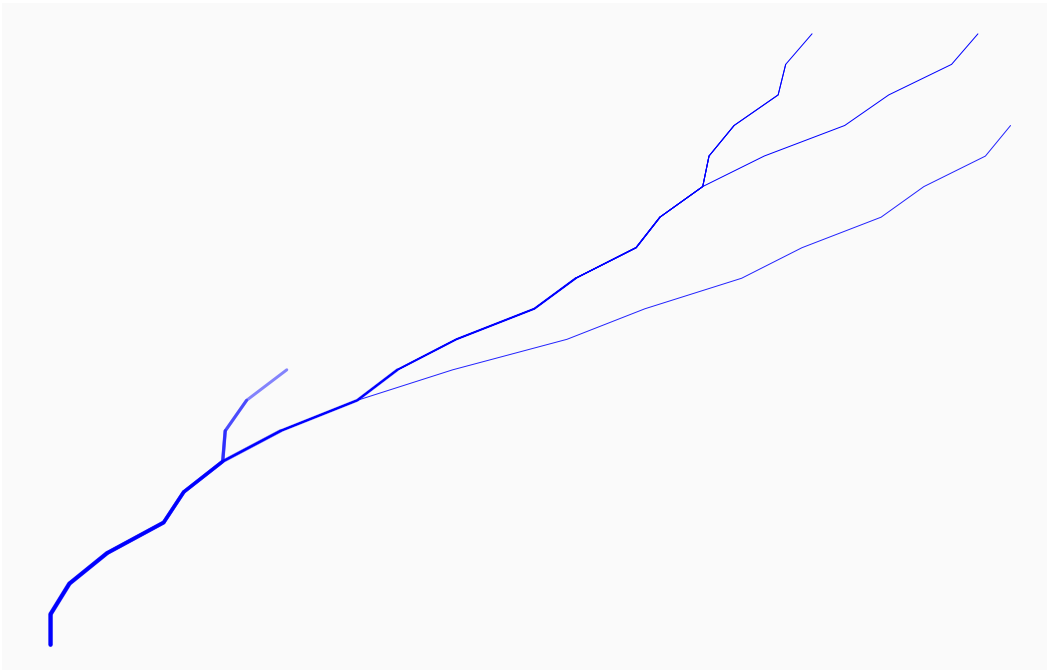
```
set.seed(1)
canvas_collatz(
  colors = c("orange"),
  n = 10
)
```



Question #4

Contrast the code below with the code shown in question 3. What is this code doing differently than the previous chunk? That is, what's different both in the code, and the corresponding result? Next, explain if it would be appropriate to include `set.seed(1)` with the code below.

```
canvas_collatz(  
  colors = c("blue"),  
  n = 1:20  
)
```



This code is producing the same result upon each render without the `set.seed()` function. The difference in code has to do with the `n` argument. Question #3's code has `n = 10` for 10 random integers, whereas this code has `n = 1:20` which I believe means the numbers 1 through 20 are being used as the starting point for the conjecture. Including `set.seed(1)` in this code is not necessary because it has a set starting point rather than a randomly selected starting point.